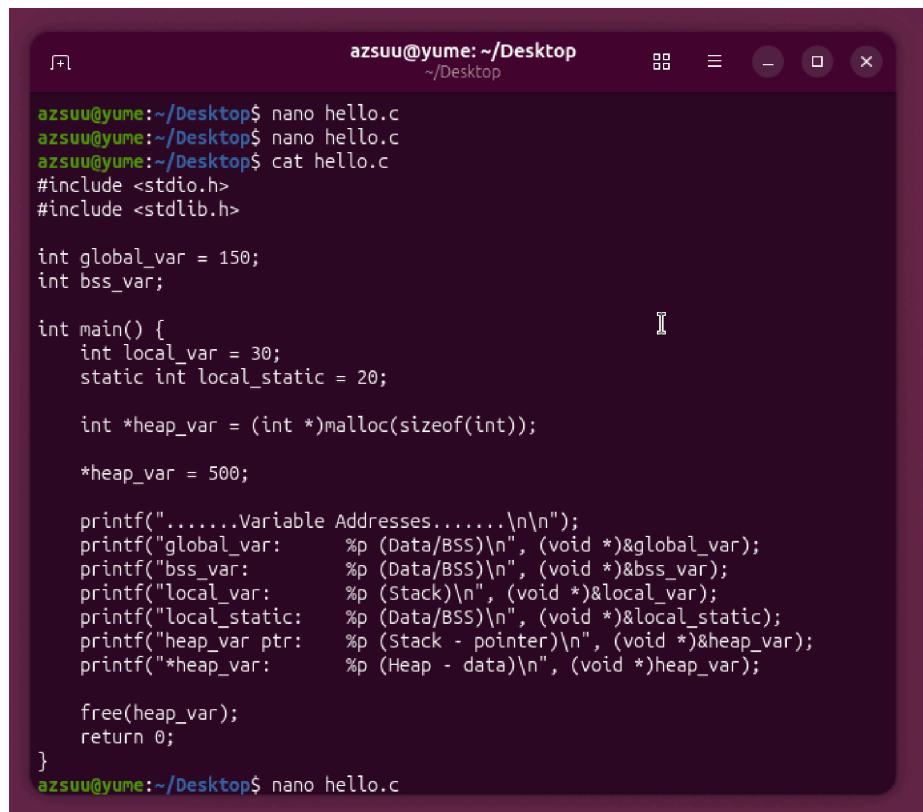


Task 4



```
azsuu@yume: ~/Desktop
azsuu@yume:~/Desktop$ nano hello.c
azsuu@yume:~/Desktop$ nano hello.c
azsuu@yume:~/Desktop$ cat hello.c
#include <stdio.h>
#include <stdlib.h>

int global_var = 150;
int bss_var;

int main() {
    int local_var = 30;
    static int local_static = 20;

    int *heap_var = (int *)malloc(sizeof(int));

    *heap_var = 500;

    printf(".....Variable Addresses.....\n\n");
    printf("global_var:      %p (Data/BSS)\n", (void *)&global_var);
    printf("bss_var:           %p (Data/BSS)\n", (void *)&bss_var);
    printf("local_var:         %p (Stack)\n", (void *)&local_var);
    printf("local_static:      %p (Data/BSS)\n", (void *)&local_static);
    printf("heap_var ptr:      %p (Stack - pointer)\n", (void *)&heap_var);
    printf("*heap_var:         %p (Heap - data)\n", (void *)heap_var);

    free(heap_var);
    return 0;
}
azsuu@yume:~/Desktop$ nano hello.c
```

Listing 2. Program to Observe Variable Addresses

This program prints the memory addresses of different variables to prove exactly where they are stored in the computer's memory (Stack, Heap, or Data segment).

Variables Created: It creates different types of variables:

global_var & bss_var: Global variables (stored in the Data/BSS segment).

local_var: A normal local variable (stored on the Stack).

local_static: A static local variable (also stored in the Data segment).

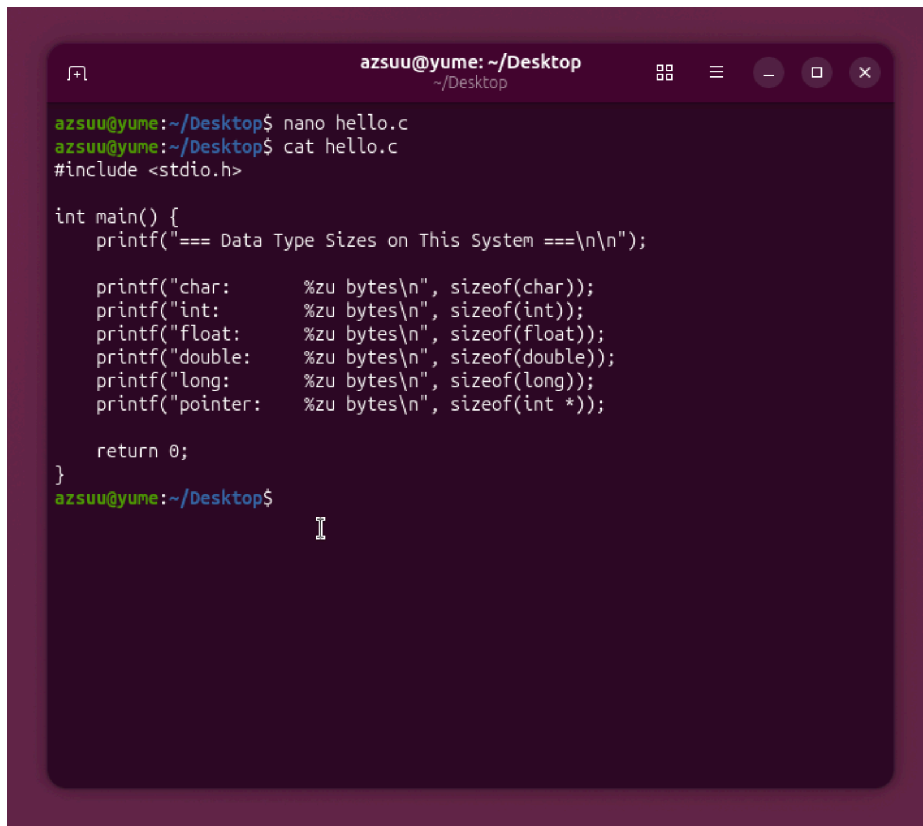
heap_var: A pointer (stored on the Stack) that points to dynamically allocated memory (stored on the Heap).

The & symbol: In the printf statements, & means "address of." It gets the memory location of the variable itself.

(void *): This is just a "cast." The %p format specifier requires a generic pointer, so this just makes the compiler happy.

free(heap_var);: This is very important! It gives the Heap memory back to the computer so it doesn't leak.

It proves that local variables live on the Stack, malloc data lives on the Heap, and global/static variables live in the Data segment by printing where they are located.

A screenshot of a terminal window with a dark purple background. The window title is 'azsuu@yume: ~/Desktop'. The terminal shows the following commands and output:

```
azsuu@yume:~/Desktop$ nano hello.c
azsuu@yume:~/Desktop$ cat hello.c
#include <stdio.h>

int main() {
    printf("=== Data Type Sizes on This System ===\n\n");

    printf("char:      %zu bytes\n", sizeof(char));
    printf("int:        %zu bytes\n", sizeof(int));
    printf("float:      %zu bytes\n", sizeof(float));
    printf("double:    %zu bytes\n", sizeof(double));
    printf("long:      %zu bytes\n", sizeof(long));
    printf("pointer:   %zu bytes\n", sizeof(int *));

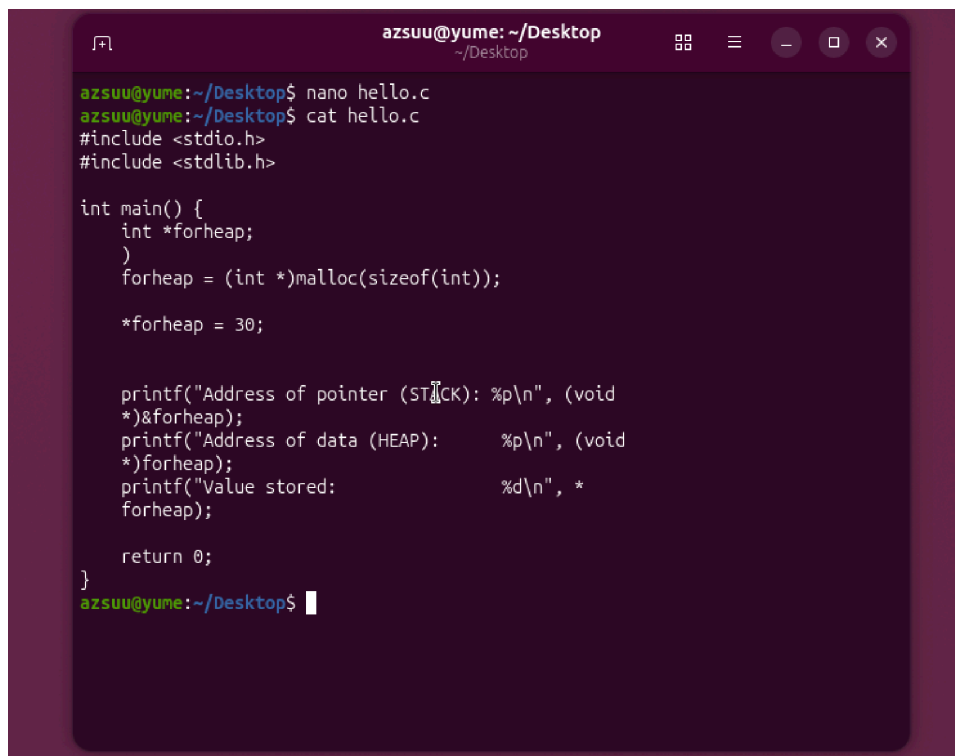
    return 0;
}
azsuu@yume:~/Desktop$
```

Listing 1. Program to Observe Data Type Sizes

This program checks and prints how much memory (in bytes) different data types use on your specific computer.

- `#include <stdio.h>`: Loads the tools needed to print text to the screen.
- `int main()`: The starting point of the program.
- `sizeof()`: This is the key command. It asks the computer, "How many bytes does this data type take up?" (e.g., `sizeof(int)`).
- `printf(...)`: Prints the results. The `%zu` is just a placeholder that gets replaced by the number `sizeof` calculates.
- `\n`: Tells the computer to go to a new line.

It asks the computer the size of `char`, `int`, `float`, `double`, `long`, and a pointer, and then prints the answers.



```
azsuu@yume: ~/Desktop
azsuu@yume:~/Desktop$ nano hello.c
azsuu@yume:~/Desktop$ cat hello.c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *forheap;
    )
    forheap = (int *)malloc(sizeof(int));

    *forheap = 30;

    printf("Address of pointer (STACK): %p\n", (void
*)&forheap);
    printf("Address of data (HEAP):      %p\n", (void
*)&forheap);
    printf("Value stored:                %d\n", *
forheap);

    return 0;
}
azsuu@yume:~/Desktop$
```

Listing 3. Correct Pointer and Heap Usage

This program is a simple demonstration of how pointers work with the **Stack** and the **Heap**.

- * `int *forheap;`: Creates a pointer variable. The pointer itself lives on the **Stack**.
- * `malloc(sizeof(int))`: Asks the computer for memory to store an integer. This new memory is created on the **Heap**.
- * `*forheap = 30;`: Takes the number `30` and stores it inside that new Heap memory.
- * `First printf (&forheap)`: Uses `&` to get the address of the pointer variable itself (proving it is on the Stack).
- * `Second printf (forheap)`: Prints the address the pointer is holding (proving it points to the Heap).
- * `Third printf (*forheap)`: Uses `*` to look inside that Heap address and print the value stored there (`30`).

It proves that the `*pointer` sits on the Stack, but the `*data` it points to sits on the Heap.